

Web Service Design and Practice

08. Database 연동

Dr. Kyudong Park

School of Information Convergence



Contents

- [데이터베이스](#)
- MySQL
- MongoDB

1. Database

- 여러 사람이 공유하여 사용할 목적으로 체계화해 통합, 관리하는 데이터의 집합
- 이러한 데이터의 집합을 제어하는 시스템
 - Database Management System = DBMS
 - RDB = Relational Database
 - Table 로 표현 가능
 - NoSQL = Not only SQL
 - 엄격한 스키마가 없는 유연성 (column 관점에서 생각해보기)
 - json 과 같은 문서 형태 = mongoDB
 - graph 를 활용 = neo4j
 - SNS 친구 관계

1. Database

- Web Server 에서 Database 란?
 - 여러 client 에서 들어오는 데이터의 안정적이고 영구적인 CRUD
 - 2차 메모리에 데이터를 저장하여 서버가 꺼지더라도 데이터의 보존
 - 보안 향상을 위한 별도의 서버
- 백엔드 개발자의 역량 측면
 - 서비스에 필요한 스키마를 설계하는 역량
 - API 호출에 대한 데이터를 가공할 때, 최적화할 수 있는 역량

Contents

- 데이터베이스
- [MySQL](#)
- MongoDB

2. MySQL

- 설치

- <https://dev.mysql.com/downloads/mysql/>

MySQL Community Downloads

MySQL Community Server

[General Availability \(GA\) Releases](#) [Archives](#) [↓](#)

MySQL Community Server 8.0.31

Select Operating System:

[Looking for previous GA versions?](#)

Recommended Download:

MySQL Installer

for Windows

All MySQL Products. For All Windows Platforms.
In One Package.

Starting with MySQL 5.6 the MySQL Installer package replaces the standalone MSI packages.



Windows (x86, 32 & 64-bit), MySQL Installer MSI [Go to Download Page >](#)

Other Downloads:

Windows (x86, 64-bit), ZIP Archive (mysql-8.0.31-winx64.zip)	8.0.31	222.3M	Download
		MD5: c9135ec4988a41a932f8db2b6661d1ed Signature	
Windows (x86, 64-bit), ZIP Archive Debug Binaries & Test Suite (mysql-8.0.31-winx64-debug-test.zip)	8.0.31	555.6M	Download
		MD5: 6ac7b4aa309f1cab01c4e25ebf614e4d7 Signature	

2. MySQL

- 데이터베이스 생성
 - Schema Name: kwic
 - Character set : utf8
 - Collation: utf8_general_ci
- customers 테이블 생성
 - id / INT / primary key / not null / auto increment
 - name / VARCHAR(45)
 - phone / VARCHAR(45)

2. MySQL

- 샘플 SQL 쿼리 실행
 - `insert into kwic.customers(id, name, phone) values(1, 'kdpark', '010-1234-1234')`
 - 실행 결과 확인
- 접속용 계정 생성 (Mysql shell 접속)
 - JS > `\sql`
 - SQL > `\connect root@localhost:3306`
 - > `create user 'dev01'@'%' identified with MYSQL_NATIVE_PASSWORD BY '1234';`
 - > `grant all privileges on kwic.* to 'dev01'@'%' with grant option;`
 - > `flush privileges;`

2. MySQL

- node.js 에서 mysql 조작하기
 - 모듈 설치
 - `$ npm install mysql`
 - 연결 체계
 - 데이터베이스에 연결
 - 쿼리를 수행
 - 결과를 받은 후
 - 연결을 종료

2. MySQL

- 조회 기능
 - mysql/sql.js : query 가 작성되는 파일
 - mysql/index.js : DB 연결을 구성하고, query 의 실행, 결과 반환을 담당하는 파일
 - routes/index.js : root 경로의 실행
 - postman 으로 체크
 - get 으로 해당 경로 요청

```
✓ EXPRESS_DATABASE_MYSQL
  ✓ bin
    JS www
  ✓ mysql
    JS index.js
    JS sql.js
  > node_modules
  > public
  ✓ routes
    ✓ api
      JS customers
      JS index.js
      JS users.js
  ✓ views
    <> index.html
  ⚙ .env
  🔒 .gitignore
  JS app.js
  📄 LICENSE
  {} package-lock.json
  {} package.json
```

2. MySQL

- DB 연결의 구성 내용이 전부 공개됨
 - 코드에 노출되면 안됨
 - dotenv 모듈의 사용
 - .env 파일에 저장된 데이터를 환경 변수로 처리할 수 있게 함
 - .env 파일은 github 등에 업로드 되지 않도록 주의

```
MYSQL_HOST = localhost
MYSQL_PORT = 3306
MYSQL_USERNAME=dev01
MYSQL_PASSWORD=1234
MYSQL_DB=kwic
MYSQL_LIMIT=10
```

2. MySQL

- 데이터의 추가 (post)
 - insert into customers set ?
 - postman 체크
 - post
 - URL : http://localhost:3000/api/customer/insert
 - Body -> raw -> JSON
 - 데이터 입력

```
{  
  "param" : {  
    "name" : "kwangwoon",  
    "phone" : "010-4321-4321"  
  }  
}
```

2. MySQL

- 데이터의 수정 (put)
 - update customers set ? where id=?
 - postman 체크
 - PUT
 - URL : http://localhost:3000/api/customer/update
 - Body -> raw -> JSON
 - 데이터 입력

```
{  
  "param" : [  
    {"phone" : "010-3333-3333"},  
    1  
  ]  
}
```

2. MySQL

- 데이터의 삭제 (delete)
 - delete from customers where id=?
 - postman 체크
 - DELETE
 - URL : <http://localhost:3000/api/customer/delete/2>

2. MySQL

- ORM 에 대해서...
 - Object-relational mapping
 - query 를 몰라도 javascript 의 객체 형태로 데이터를 처리할 수 있음
 - 장점과 단점을 명확히 파악하고 사용할 것
 - 장점 : 객체지향 프로그래밍 원칙과도 부합, 비즈니스 로직에 집중, 데이터베이스 종속 완화, 가독성 증가
 - 단점 : 설계에 따른 성능 최적화 어려움, 서브쿼리를 표현할 수 없음
 - SQL 은 반드시 할 수 있어야 함
- knex.js 와 같은 쿼리빌더를 활용하는 것 또한 대안
 - 추상화의 정도 : SQL < 쿼리빌더 < ORM
 - 예시
 - `knex('users').insert({ name: ' kdpark', age: 30 })`
 - `knex('users').update({ age: 21 }).where('id', 2)`

Contents

- 데이터베이스
- MySQL
- [MongoDB](#)

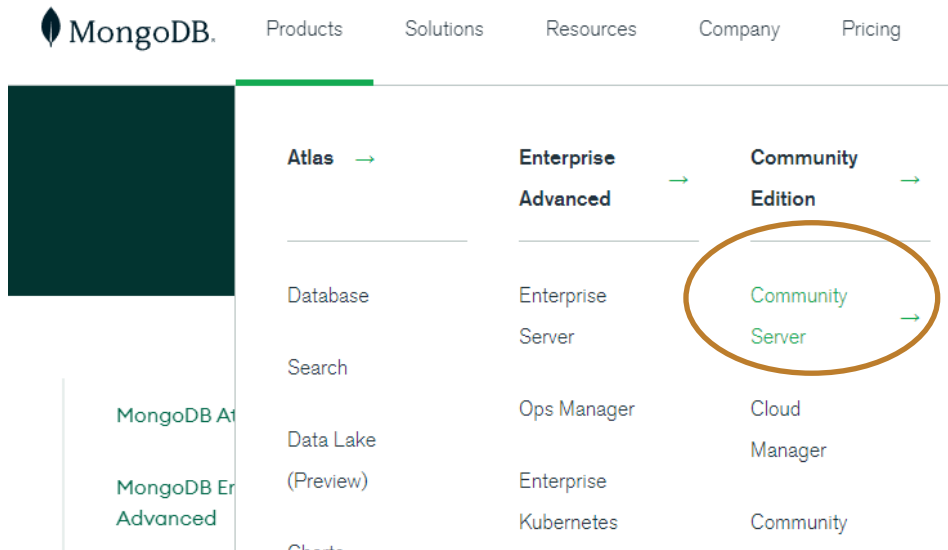
3. MongoDB

- mysql 이 table 형태 (행, 열) 로 데이터의 저장하는 반면,
mongodb 는 collection 과 document 를 활용하여 데이터를 저장함
- 비교

SQL (MySQL)	NoSQL (MongoDB)
규칙에 맞는 데이터 입력	자유로운 데이터 입력
테이블 간 JOIN 지원	컬렉션 간 JOIN 미지원
안정성, 일관성	확장성, 가용성
Table, Row, Column	Collection, Document, Field

3. MongoDB

- 설치
 - <https://www.mongodb.com/>
 - 회원가입 필수



3. MongoDB

- 관리자 계정 추가
 - `$ mongo`
 - `> use admin`
 - `> db.createUser( {user : "root", pwd : "1234", roles : [ "root" ]})`
- 데이터베이스 생성
 - MongoDB COMPASS (windows 는 자동 설치, mac 은 별도 설치 필요)
 - Database name: kwic
 - Collection name: customers

3. MongoDB

- 실습용 데이터 추가
 - Add data > insert documents

```
{  
  "name" : "kdpark",  
  "phone" : "010-1234-1234"  
}
```

3. MongoDB

- Mongoose
 - node.js 에서 MongoDB 에 접속하고 CRUD 처리를 도와주는 ODM (object-document mapping)
 - 설치
 - `$ npm install mongoose`
 - 연결 형식
 - `mongoose.connect('mongodb://username:password@host:port/database', options)`

3. MongoDB

- 연결 생성
 - /mongoose/index.js
- 스키마생성
 - /mongoose/schemas/customer.js
- 모델 생성
 - mongoose.model 함수를 사용하여 생성

3. MongoDB

- 문서 조회
 - Model.find()
 - filter 및 파라미터 조건 등을 활용하여 다양하게 조회 가능
 - `await Customer.find( { name: 'kdpark' } ).exec();`
 - Model.findById()
 - 주어진 id 값에 해당하는 문서를 하나 가져옴
 - `await Customer.findById(id).exec();`
 - Model.findOne()
 - find 와 같지만, 하나의 문서만 결과로 반환함
 - `await Customer.findOne( {name: 'kd' } ).exec();`

3. MongoDB

- 문서 추가

- 단일 데이터 추가

- ```
await Customer.create({
 name: "Customer2",
 phone: "010-2222-2222",
 });
```

- 복수 데이터 추가

- ```
await Customer.create( [ {  
    name: "Customer2",  
  }, {  
    name: "Customer3",  
  } ] );
```


3. MongoDB

- 문서 수정

- MyModel.findByIdAndUpdate()

- 조회 후 수정
 - `await Customer.findByIdAndUpdate( id, { phone: '010-2233-2233' } );`

- MyModel.updateMany()

- 조회된 모든 문서를 수정
 - `await Customer.updateMany( {name: /kd/ }, { phone: '010-2233-2233' } );`

- MyModel.updateOne()

- 조회된 결과 중 첫 번째 문서를 수정함
 - `await Customer.updateOne({name: 'kdpark' }, { phone: '010-2233-2233' } );`

3. MongoDB

- 문서 삭제
 - Model.deleteMany()
 - 조건에 해당하는 모든 문서를 삭제
 - `await Customer.deleteMany({ name: /kd/ });`
 - Model.deleteOne()
 - 조회된 결과 중 첫번째 문서를 삭제
 - `await Customer.deleteOne( { name: 'kdpark' } );`

실습용 코드

- mysql
 - https://github.com/kw-ic-web/express_database_mysql
- mongoDB
 - https://github.com/kw-ic-web/express_database_mongo
- 코드 검토용으로만 활용 권장하며, 눈으로만 훑으면 학습되지 않습니다.

Thank you

Q & A